
RT-Comops Backend

Cahier de conception

Architecture Multi-Tenant — ConceptionKernel V3

Cahier de conception .

1 kernel-core

1.1 Description du core

Le **kernel-core** constitue la colonne vertébrale technique de la plateforme. Il ne contient aucune logique métier fonctionnelle au sens applicatif, mais fournit l'ensemble des mécanismes transverses sur lesquels s'appuient tous les autres cores : gestion du cycle de vie des tenants, traçabilité immuable, publication fiable d'événements via le patron *outbox transactionnel*, et sécurité des jetons d'accès.

Toute requête traitée par le système transite nécessairement par les composants de ce core (filtres, contexte réactif, routage de connexion), ce qui en fait le seul point d'entrée garanti pour les préoccupations d'infrastructure partagée.

1.2 Rôle principal

Rôle principal : Garantir l'isolation stricte des tenants, la traçabilité immuable de toutes les opérations, et la publication transactionnellement sûre des événements domaine vers le bus de messages, sans introduire de couplage fonctionnel entre les cores métier.

1.3 Rôles secondaires

1. Provisionnement des tenants

Création et activation des schémas PostgreSQL, exécution des migrations Liquibase dans le schéma du nouveau tenant, et enregistrement dans le registre central (`TenantRegistry`).

2. Routage réactif des connexions

Le composant `TenantRoutingConnectionFactory` lit l'identifiant tenant depuis le contexte Reactor et dirige chaque opération R2DBC vers le schéma PostgreSQL approprié, sans configuration explicite dans le code applicatif.

3. Filtrage et propagation du contexte tenant

Le `TenantWebFilter` intercepte chaque requête HTTP, valide le header `X-Tenant-Id`, et injecte l'identifiant tenant dans le contexte Reactor pour toute la chaîne réactive.

4. Outbox transactionnel

Toute écriture d'événement domaine est stockée dans `kernel.outbox_event` dans la même transaction que l'agrégat concerné. Un planificateur asynchrone (`OutboxRelayScheduler`) publie ensuite ces événements vers Kafka, garantissant la cohérence au moins une fois (*at-least-once delivery*).

5. Piste d'audit immuable

L'entité `AuditTrail` est une table en insertion seule : aucune mise à jour ni suppression n'est autorisée. Elle enregistre chaque action significative avec l'acteur, l'heure, l'adresse IP et le différentiel de données.

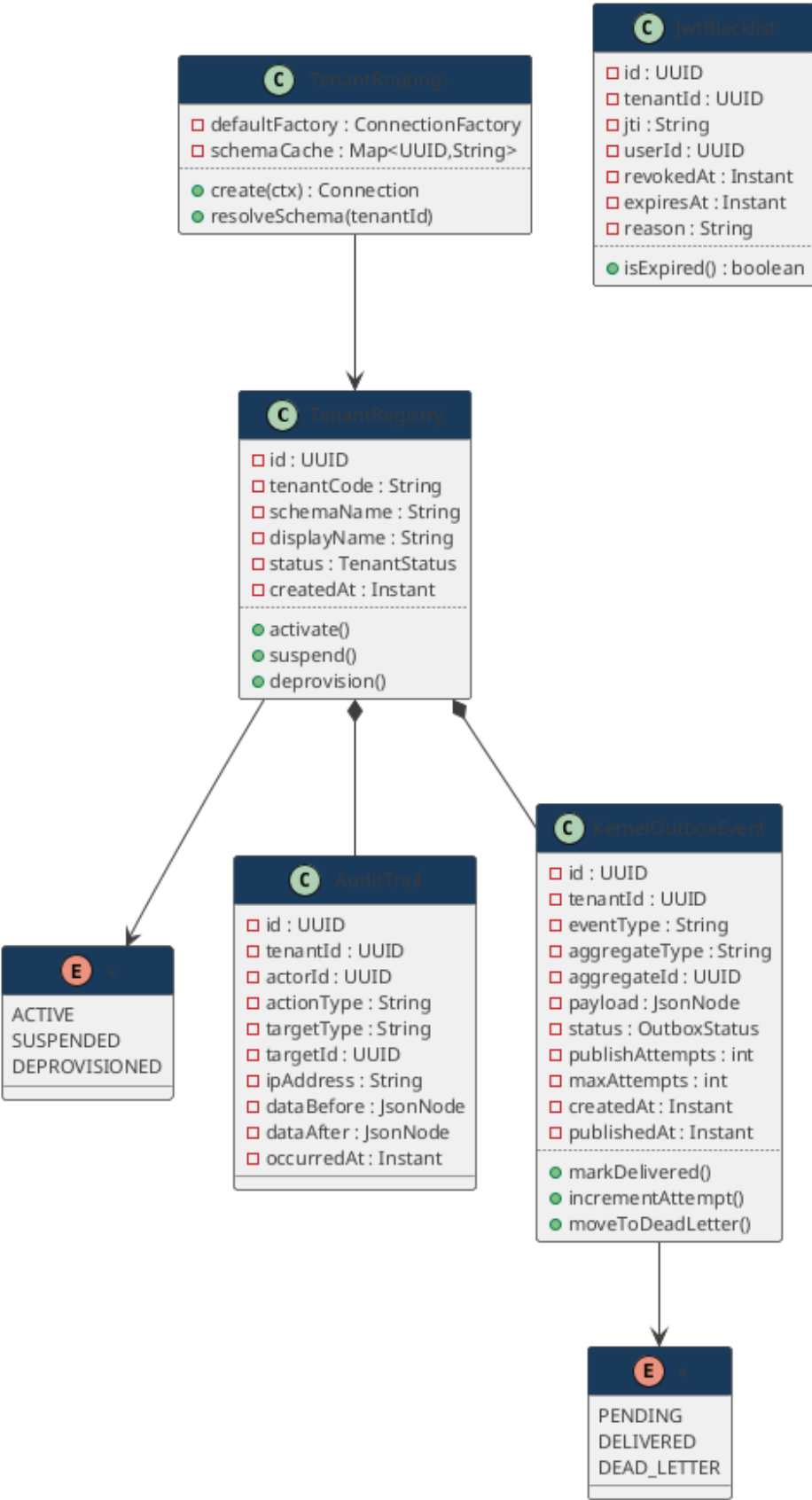
6. Gestion de la liste noire JWT

Les jetons révoqués (déconnexion, rotation forcée) sont inscrits dans `JwtBlacklist` et vérifiés systématiquement lors de chaque validation de token avant son expiration naturelle.

7. Supervision et observabilité

Exposition des métriques Micrometer / Prometheus relatives à l'état de l'outbox (nombre d'événements en attente, âge du plus ancien, dead-letter), et des health checks sur le cycle de vie des tenants.

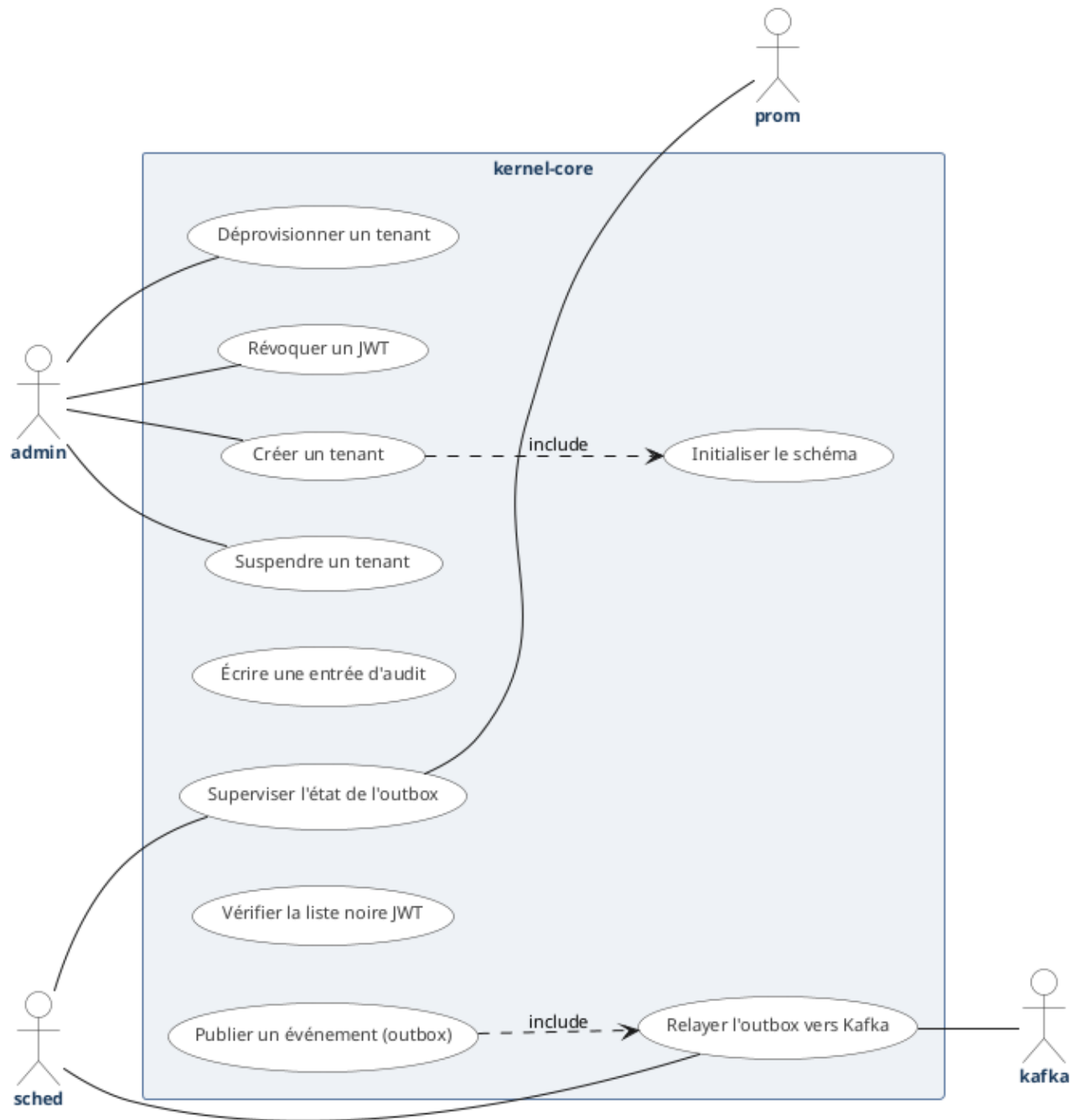
Diagramme de classes — kernel-core



Invariants métier du kernel-core :

- `tenantCode` est globalement unique (toutes bases confondues).
- `schemaName` est globalement unique.
- `status` ne peut régresser de `DEPROVISIONED` vers aucun autre état.
- `AuditTrail` est en insertion seule — aucune mise à jour ni suppression.
- `publishAttempts` ≥ 0 et `maxAttempts` > 0 .
- Un événement `DEAD_LETTER` n'est jamais re-tenté automatiquement.

Diagramme des cas d'utilisation — kernel-core



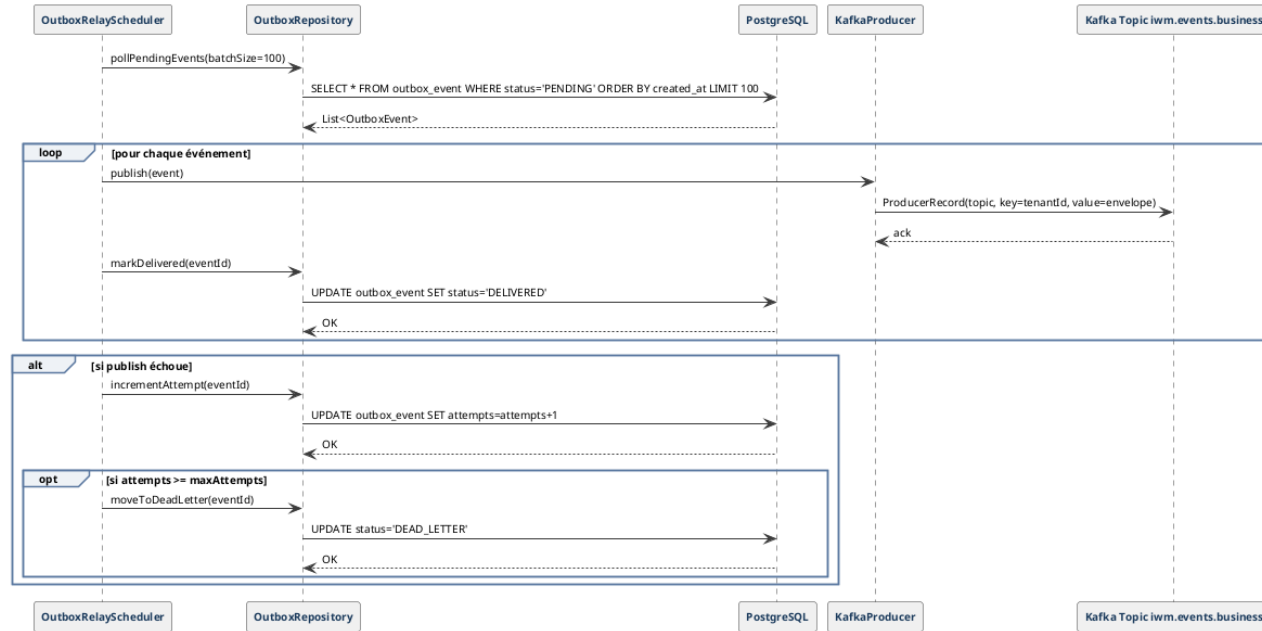
1.3.1 DS-K-01 : Provisionnement d'un nouveau tenant

Ce diagramme décrit la séquence complète d'activation d'un nouveau tenant, depuis la validation du **BusinessActor** jusqu'à la disponibilité opérationnelle du schéma.

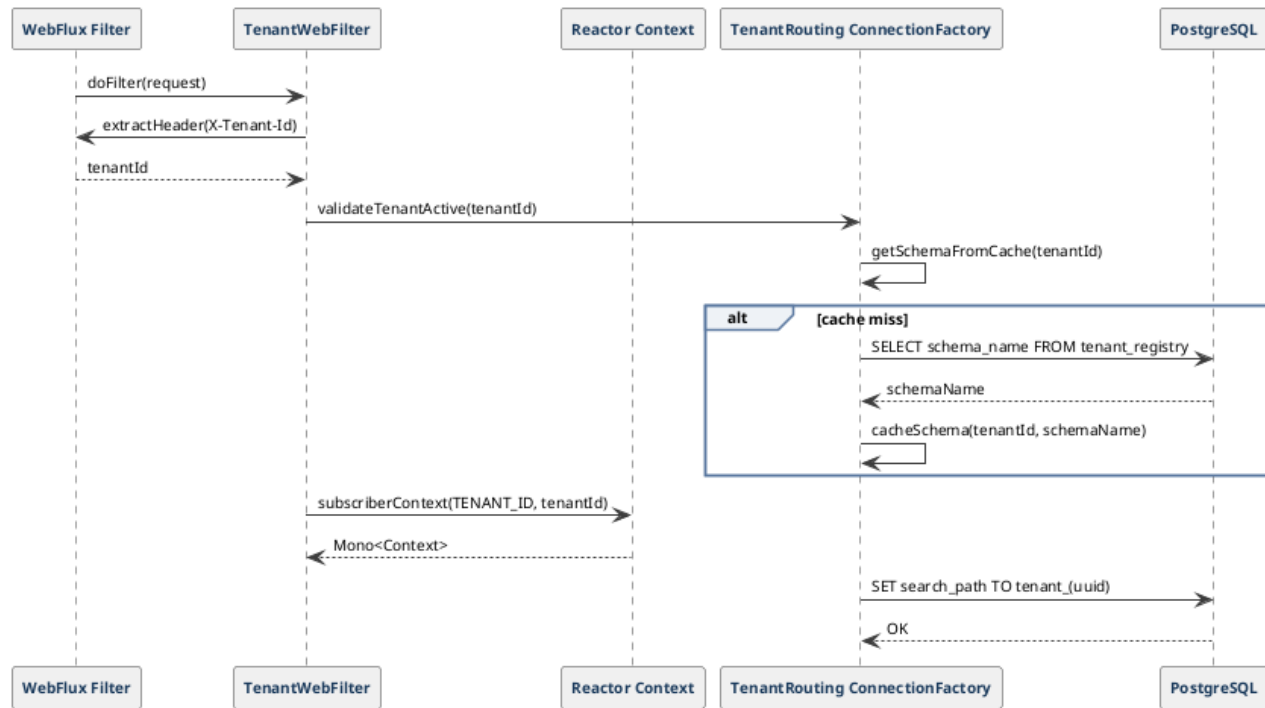
DS-K-01 — Provisionnement d'un nouveau tenant



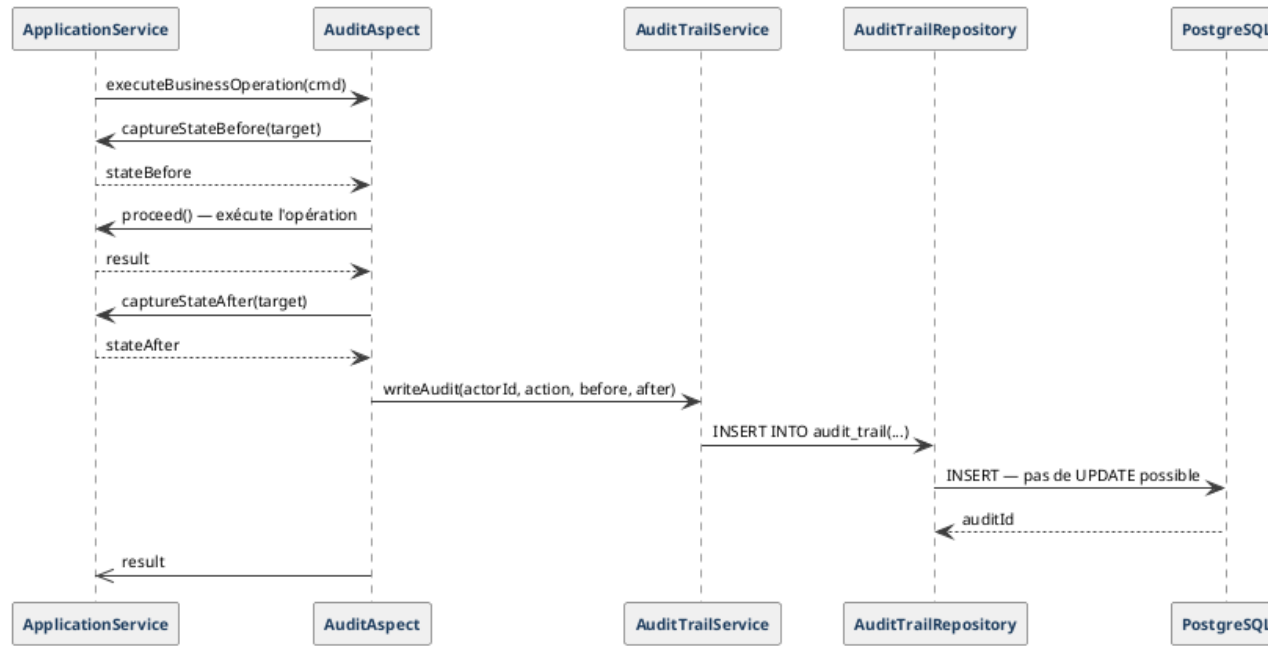
DS-K-02 — Relais Outbox vers Kafka



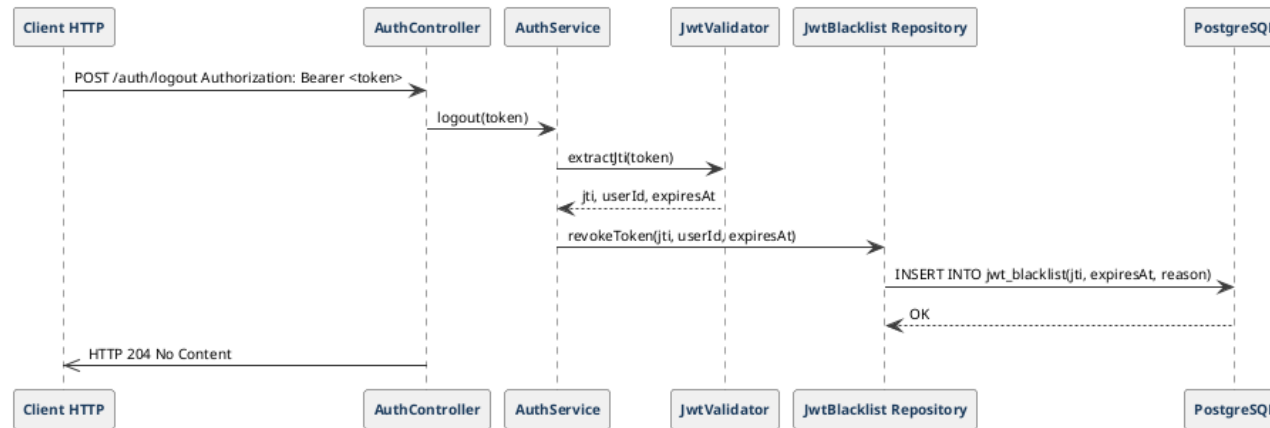
DS-K-03 — Routage réactif vers le schéma tenant



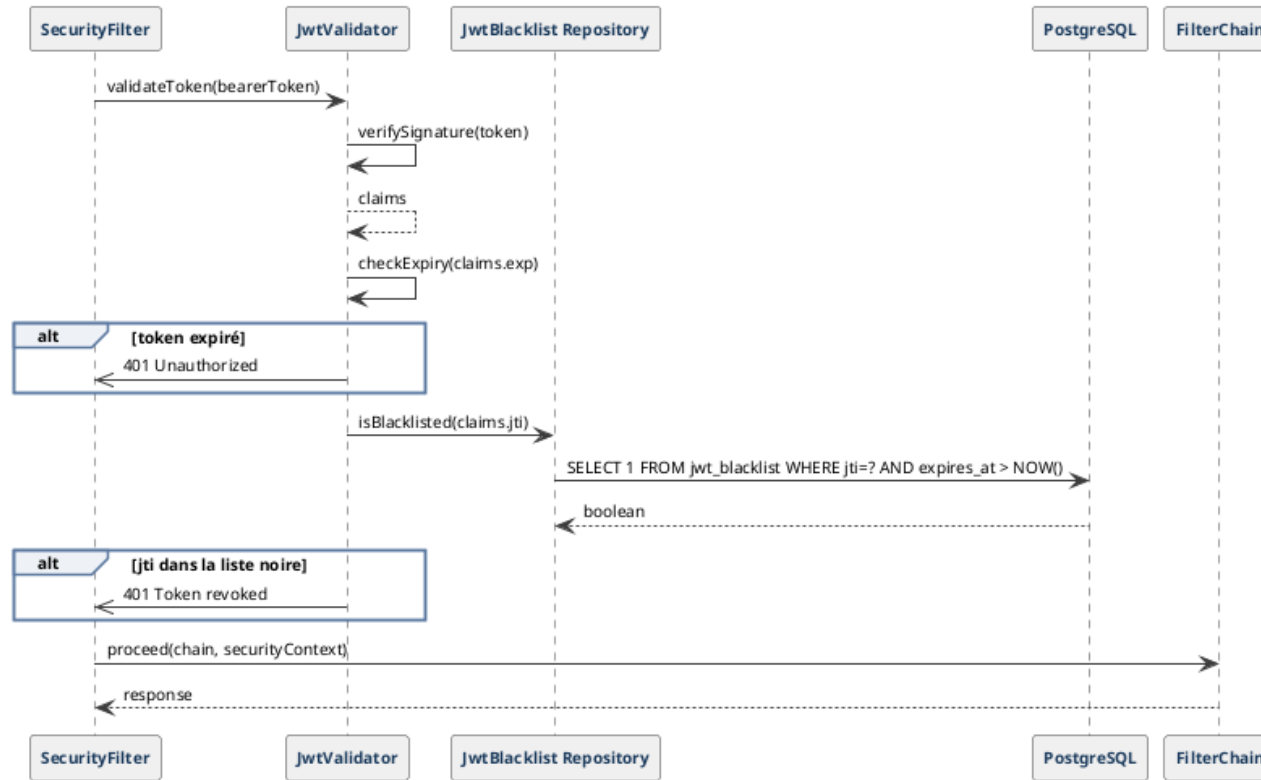
DS-K-04 — Écriture de la piste d'audit (pattern aspect)



DS-K-05 — Révocation JWT et inscription en liste noire



DS-K-06 — Validation JWT avec contrôle de la liste noire



DS-K-08 — Supervision de l'outbox via health check

